Java Class 4 — Loops

🎯 Learning Objectives

By the end of this class, students should be able to:

✓ Understand why loops are required
✓ Understand repetition in programming
✓ Use `for` loop
✓ Understand initialization, condition, and update
✓ Use `while` loop
✓ Use `do-while` loop
✓ Understand infinite loops
✓ Use `break`
✓ Use `continue`
✓ Create counting programs
✓ Calculate sums using loops
✓ Print tables using loops
✓ Solve basic loop-based problems
✓ Understand nested loops at a basic level

---

1. Quick Revision

Start with a simple problem.

Suppose we want to print:

```
Hello
Hello
Hello
Hello
Hello
```

Without a loop:

```
System.out.println("Hello");
System.out.println("Hello");
System.out.println("Hello");
System.out.println("Hello");
System.out.println("Hello");
```

Ask students:

> What if we want to print it 1,000 times?

This introduces the need for **loops**.

---

2. What Is a Loop?

A **loop** is used to execute the same block of code repeatedly as long as a condition is satisfied.

Simple idea:

```
Start
  ↓
Condition?
  ↓
```

```
Yes → Execute code
  ↓
Update
  ↓
Condition?
  ↓
...
  ↓
No
  ↓
Stop
```

---

3. First Loop — `for` Loop

Syntax:

```
for (initialization; condition; update) {
    // code
}
```

Example:

```
for (int i = 1; i <= 5; i  ) {
    System.out.println("Hello");
}
```

Output:

```
Hello
Hello
Hello
Hello
Hello
```

---

4. Understand the Three Parts

This is **very important**.

```
for (int i = 1; i <= 5; i  )
```

Part 1 — Initialization

```
int i = 1;
```

Where should the loop start?

Part 2 — Condition

```
i <= 5
```

When should the loop continue?

Part 3 — Update

```
i
```

How should the value change?

So:

```
i = 1
 ↓
condition true
 ↓
```

```
print
 ↓
i
 ↓
i = 2
 ↓
condition true
 ↓
print
...
 ↓
i = 6
 ↓
condition false
 ↓
STOP
```

---

5. Print Numbers 1 to 10

```
for (int i = 1; i <= 10; i  ) {
    System.out.println(i);
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

---

6. Print 10 to 1

```
for (int i = 10; i >= 1; i--) {
    System.out.println(i);
}
```

Output:

```
10
9
8
7
6
5
4
3
2
1
```

This helps students understand that loops don't always have to increase.

---

7. Print Even Numbers

```
for (int i = 2; i <= 20; i  = 2) {
    System.out.println(i);
}
```

Output:

```
2
4
6
8
10
12
14
16
18
20
```

Explain:

```
i  = 2
```

is the same as:

```
i = i   2;
```

---

8. Print Odd Numbers

```
for (int i = 1; i <= 20; i  = 2) {
    System.out.println(i);
}
```

---

9. Sum of Numbers

Ask:

> Find the sum of numbers from 1 to 10.

We can use a variable:

```
int sum = 0;

for (int i = 1; i <= 10; i  ) {
    sum = sum   i;
}

System.out.println(sum);
```

Output:

```
55
```

Explain the changing value:

```
sum = 0

i = 1 → sum = 1
i = 2 → sum = 3
i = 3 → sum = 6
i = 4 → sum = 10
...
i = 10 → sum = 55
```

This is an important programming concept: **accumulator variable**.

---

10. Multiplication Table

```
Scanner sc = new Scanner(System.in);

System.out.print("Enter a number: ");
```

```
int n = sc.nextInt();

for (int i = 1; i <= 10; i  ) {
    System.out.println(n   " x "   i   " = "   (n * i));
}
```

For input:

```
5
```

Output:

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
...
5 x 10 = 50
```

This combines:

Scanner variable loop calculation output

---

11. `while` Loop

The second important loop is `while`.

Syntax:

```
while (condition) {
    // code
}
```

Example:

```
int i = 1;

while (i <= 5) {
    System.out.println(i);
    i  ;
}
```

Output:

```
1
2
3
4
5
```

---

12. `for` vs `while`

`for`

Generally useful when we know approximately how many times we want to repeat.

```
for (int i = 1; i <= 10; i  ) {
    System.out.println(i);
}
```

`while`

Useful when repetition depends more directly on a condition.

```
while (age < 18) {
    // ...
```

```
}
```

Don't make students memorize a rigid rule. Explain that **both can often solve the same problem**.

---

13. Important: Infinite Loop

Consider:

```
int i = 1;

while (i <= 5) {
    System.out.println(i);
}
```

What's wrong?

`i` never changes.

Therefore:

```
1
1
1
1
1
1
1
...
```

The loop never ends.

Correct:

```
int i = 1;

while (i <= 5) {
    System.out.println(i);
    i  ;
}
```

Explain:

> Every loop must have a way to eventually make its condition false.

---

14. `do-while` Loop

Syntax:

```
do {
    // code
} while (condition);
```

Example:

```
int i = 1;

do {
    System.out.println(i);
    i  ;
} while (i <= 5);
```

Output:

```
1
2
```

```
3
4
5
```

---

15. Important Difference: `while` vs `do-while`

This is a very common question.

`while`

Condition is checked **before** execution.

```
int i = 10;

while (i <= 5) {
    System.out.println(i);
}
```

Output:

```
Nothing
```

`do-while`

Code executes **at least once**.

```
int i = 10;

do {
    System.out.println(i);
} while (i <= 5);
```

Output:

```
10
```

Remember:

> **while → check first**
>
> **do-while → execute first**

---

16. `break`

`break` is used to immediately exit a loop.

Example:

```
for (int i = 1; i <= 10; i  ) {

    if (i == 5) {
        break;
    }

    System.out.println(i);
}
```

Output:

```
1
2
3
4
```

When `i == 5`, the loop stops.

---

17. `continue`

`continue` skips the current iteration and moves to the next iteration.

Example:

```
for (int i = 1; i <= 5; i  ) {

    if (i == 3) {
        continue;
    }

    System.out.println(i);
}
```

Output:

```
1
2
4
5
```

Difference:

`break` → **stop the entire loop**

`continue` → **skip current iteration**

---

18. Loop `if`

Now combine Class 3 and Class 4.

Print numbers from 1 to 20 that are divisible by 3:

```
for (int i = 1; i <= 20; i  ) {

    if (i % 3 == 0) {
        System.out.println(i);
    }
}
```

Output:

```
3
6
9
12
15
18
```

This is where students start thinking like programmers.

---

19. Find Factorial

Example:

```
5! = 5 × 4 × 3 × 2 × 1
   = 120
```

Java:

```
int n = 5;
int factorial = 1;

for (int i = 1; i <= n; i  ) {
    factorial = factorial * i;
}

System.out.println(factorial);
```

Output:

```
120
```

---

20. Count Digits

Introduce a slightly more logical problem.

For:

```
12345
```

Number of digits = `5`.

One approach:

```
int n = 12345;
int count = 0;

while (n > 0) {
    n = n / 10;
    count  ;
}

System.out.println(count);
```

Output:

```
5
```

This also prepares students for future number-based problems.

---

21. Nested Loops — Basic Introduction

A loop inside another loop is called a **nested loop**.

Example:

```
for (int i = 1; i <= 3; i  ) {

    for (int j = 1; j <= 3; j  ) {
        System.out.println(i   " "   j);
    }
}
```

Explain:

```
i = 1
   j = 1
   j = 2
   j = 3

i = 2
   j = 1
   j = 2
   j = 3
```

```
i = 3
   j = 1
   j = 2
   j = 3
```

Don't go too deep into patterns yet. That can be a separate practice session.

---

22. First Pattern Program

Introduce a simple pattern:

```
*
*
*
*
*
```

```
for (int i = 1; i <= 5; i  ) {
    System.out.println("*");
}
```

Then:

```
*****
```

```
for (int i = 1; i <= 5; i  ) {
    System.out.print("*");
}
```

This is a good moment to explain:

`println`

Moves to the next line.

`print`

Stays on the same line.

---

23. Nested Pattern

Now:

```
*****
*****
*****
*****
*****
```

Code:

```
for (int i = 1; i <= 5; i  ) {

    for (int j = 1; j <= 5; j  ) {
        System.out.print("*");
    }

    System.out.println();
}
```

Explain:

- Outer loop → rows
- Inner loop → columns

This concept becomes extremely useful later.

---

24. Practice Questions

Level 1

1. Print numbers from 1 to 100.
2. Print numbers from 100 to 1.
3. Print even numbers from 1 to 100.
4. Print odd numbers from 1 to 100.
5. Print the multiplication table of a number.
6. Find the sum of numbers from 1 to 100.

Level 2

7. Find the factorial of a number.
8. Count the digits of a number.
9. Find the sum of digits.

Example:

```
1234 → 10
```

10. Reverse a number.

```
1234 → 4321
```

11. Check whether a number is palindrome.

```
121 → Palindrome
123 → Not Palindrome
```

12. Check whether a number is prime.

Level 3

13. Print all prime numbers from 1 to 100.
14. Find the largest digit in a number.
15. Find the smallest digit in a number.
16. Count even and odd digits.
17. Print this pattern:

```
*
**
***
****
*****
```

18. Print:

```
1
12
```

```
123
1234
12345
```

📝 Class 4 Assignment

Give students these **10 compulsory questions**:

```
1. Print 1 to 50.
2. Print 50 to 1.
3. Print all even numbers from 1 to 100.
4. Print the table of any number entered by the user.
5. Find the sum of 1 to N.
6. Find factorial of N.
7. Reverse a number.
8. Count digits of a number.
9. Check whether a number is palindrome.
10. Check whether a number is prime.
```

What Students Should Know After Class 4

By the end of this class, students should be able to:

✓ Understand why loops are used
✓ Understand repetition
✓ Use `for` loop
✓ Understand initialization, condition and update
✓ Use `while` loop
✓ Use `do-while` loop
✓ Know the difference between `while` and `do-while`
✓ Understand infinite loops
✓ Use `break`
✓ Use `continue`
✓ Use loops with `if-else`
✓ Calculate sums and factorials
✓ Generate multiplication tables
✓ Reverse numbers
✓ Count digits
✓ Understand basic nested loops
✓ Understand `print()` vs `println()`
✓ Create basic star/number patterns

🚀 Class 5 — Arrays

The progression should now be:

Class 1: Java setup first program
↓
Class 2: Variables Data Types String Scanner
↓
Class 3: Operators `if-else`
↓
Class 4: Loops
↓
Class 5: Arrays in Java
↓
Class 6: Methods

↓

Class 7: Strings in depth

↓

Class 8: OOP fundamentals (Class, Object, Constructor)